

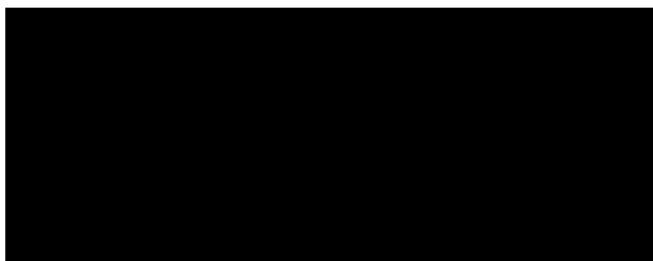
电子科技大学

计算机专业类课程

实验报告

课程名称：计算机网络编程

学 院：计算机学院



指导教师：向渝

日 期： 2025 年 12 月 14 日

电子科技大学

实验报告

实验四

一、实验名称：无连接的 TIME 服务器软件的实现

二、实验学时：4

三、实验内容和目的：

掌握服务端软件的工作原理。

掌握无连接的 TIME 服务器软件的编程实现方法。

四、实验原理：

使用指定编程工具编写无连接的 TIME 服务器软件的源代码。

使用指定编程工具对代码进行编译和调试，生成执行程序。

在 PC 机上执行程序，检验程序能提供的 TIME 服务的功能。

五、实验步骤：

1.总体设计思路

本实验基于 UDP 协议，使用 C 语言实现一个无连接的 TIME 服务器程序。服务器遵循 TIME 协议（RFC 868），在接收到客户端的时间请求后，返回一个以 1900 年 1 月 1 日 00:00:00（UTC）为起点的 32 位无符号整型时间值。

程序整体工作流程为：

创建 UDP 套接字 → 绑定 TIME 服务端口 → 循环接收客户端请求 → 获

取当前时间 → 发送 TIME 响应数据

2.创建 UDP 套接字

服务器程序通过 `socket()` 创建一个基于 IPv4 的 UDP 套接字 (`SOCK_DGRAM`)，用于接收客户端的时间请求并返回响应数据。

套接字创建失败时，程序立即输出错误信息并终止执行，保证服务器不会在异常状态下运行。

```
//Create UDP socket
sockfd=socket(AF_INET,SOCK_DGRAM,0);
if (sockfd<0){
    perror("Socket creation failed");
    exit(EXIT_FAILURE);
}
```

3.服务器地址结构体初始化与端口绑定

使用 `sockaddr_in` 结构体对服务器地址进行初始化，设置地址族为 `AF_INET`，IP 地址为 `INADDR_ANY`，端口号设置为 TIME 服务的标准端口 37。随后调用 `bind()` 将 UDP 套接字绑定到指定端口，使服务器能够接收来自任意客户端的时间请求。

```
//init server address struct
memset(&server_addr,0,sizeof(server_addr));
server_addr.sin_family=AF_INET;
server_addr.sin_addr.s_addr=INADDR_ANY;//echo all network
server_addr.sin_port=htons(TIME_PORT);

//bind socket to the port
if(bind(sockfd,(const struct sockaddr *)&server_addr,sizeof(server_addr))<0){
    perror("Bind failed");
    close(sockfd);
    exit(EXIT_FAILURE);
}
printf("TIME server listening on UDP port %d...\n",TIME_PORT);
```

4.TIME 时间数据生成逻辑设计

为生成符合 TIME 协议的数据格式，程序设计了 `get_time_seconds()` 函数：调用 `time()` 获取当前 Unix 时间；加上 TIME 协议与 Unix 纪元之间的时间差；使用 `htonl()` 转换为网络字节序，确保跨平台传输的正确性。

```
//Get time message from TIME
time_data=get_time_seconds();
```

5.接收客户端请求并返回 TIME 响应

服务器在主循环中调用 `recvfrom()` 接收客户端发送的 UDP 数据报。每当收到客户端请求，服务器输出客户端的 IP 地址和端口号作为连接日志，并调用 `sendto()` 将当前时间数据返回给对应客户端。由于 UDP 协议为无连接通信，服务器无需维护客户端连接状态，即可同时为多个客户端提供 TIME 服

务。

```
//Loop to handle client requests
while(1){
    //Receive client data
    ssize_t recv_len=recvfrom(sockfd,buffer,BUFFER_SIZE,0,(struct sockaddr
*)&client_addr,&client_len);
    if(recv_len<0){
        perror("Recvfrom failed");
        continue;
    }
    //print client message
    printf("Received request from:%s:
%d\n",inet_ntoa(client_addr.sin_addr),ntohs(client_addr.sin_port));
    //send time data to client
    ssize_t send_len=sendto(sockfd,&time_data,sizeof(time_data),0,(struct sockaddr
*)&client_addr,client_len);
    if(send_len<0){
        perror("Sendto failed");
    }else {
        printf("Sent time data:%u seconds since 1900.01.01 00:00:00
UTC\n",ntohl(time_data));
    }
}
```

六、实验数据及结果分析:

1.服务器启动与监听状态分析

编译并运行 TIME 服务器程序后,服务器成功绑定 UDP 37 号端口,并在终端中输出“TIME server listening on UDP port 37”的提示信息,表明服务器已正常启动并进入服务状态。

```
yar@2023080910002:~/computer_network$ sudo ./time_server_udp
[sudo] yar 的密码:
TIME server listening on UDP port 37...
```

2.客户端请求接收情况分析

在客户端程序向服务器发送 TIME 请求后,服务器能够正确接收到客户端的 UDP 数据报,并在终端中输出客户端的 IP 地址和端口号信息,说明服务器可以同时响应来自不同客户端的请求。

```
Received request from:127.0.0.1:60367
```

3.TIME 数据响应正确性分析

服务器返回的数据为 32 位无符号整数,表示自 1900 年 1 月 1 日 00:00:00 (UTC) 起的秒数。服务器端输出的时间数值与客户端接收到的时间数据一致,表明时间计算与发送过程正确完成。

```
Received request from:127.0.0.1:60367
Sent time data:3974692846 seconds since 1900.01.01 00:00:00 UTC
```

4.UDP 无连接服务特性分析

本实验中，TIME 服务器未维护任何客户端连接状态，仅根据接收到的 UDP 数据报返回时间数据，体现了 UDP 无连接通信模型的特点。实验结果表明，该实现方式结构简单、响应迅速，适合 TIME 等小数据量、低开销的网络服务场景。